
Day 5. Transformer

Samsung AI Exprt

June 9, 2025

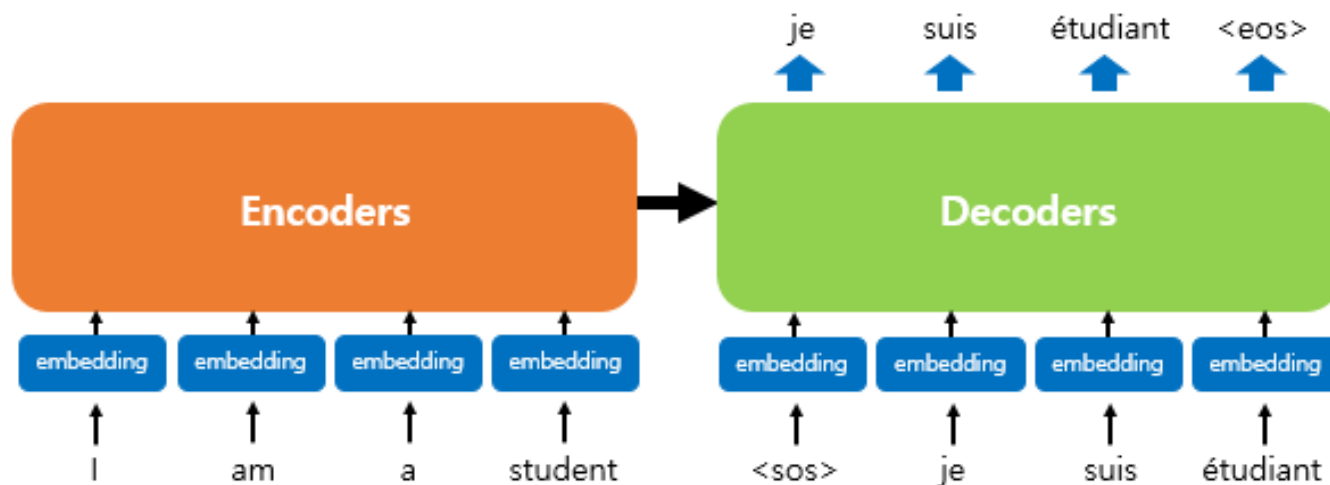
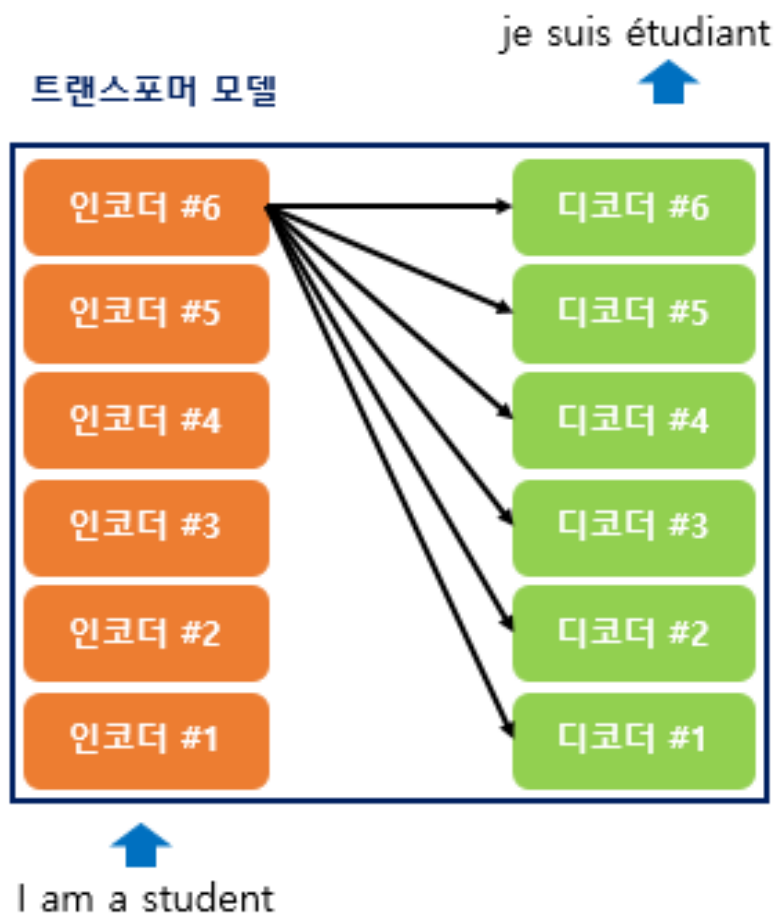
Seoul National University

Dong Hyeok Lee



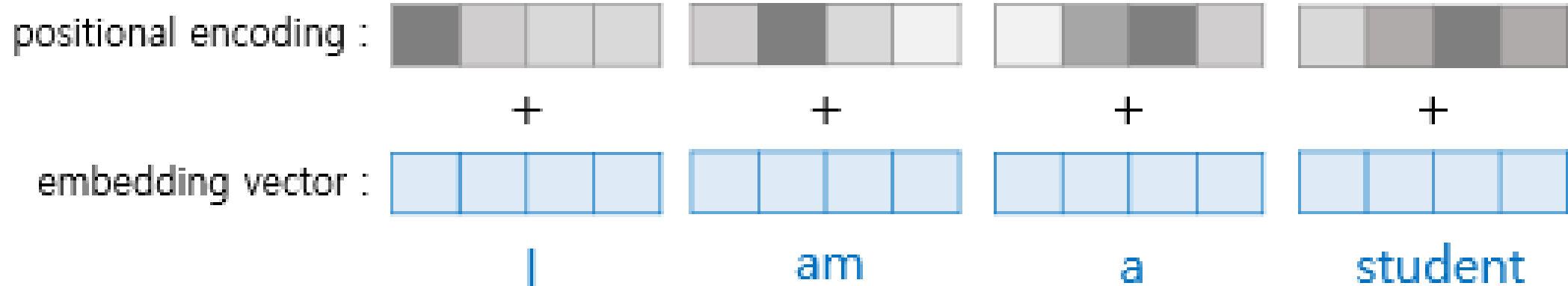
Transformer Implementation

- RNN을 사용하지 않고 어텐션(Attention)만을 사용한 인코더-디코더 구조



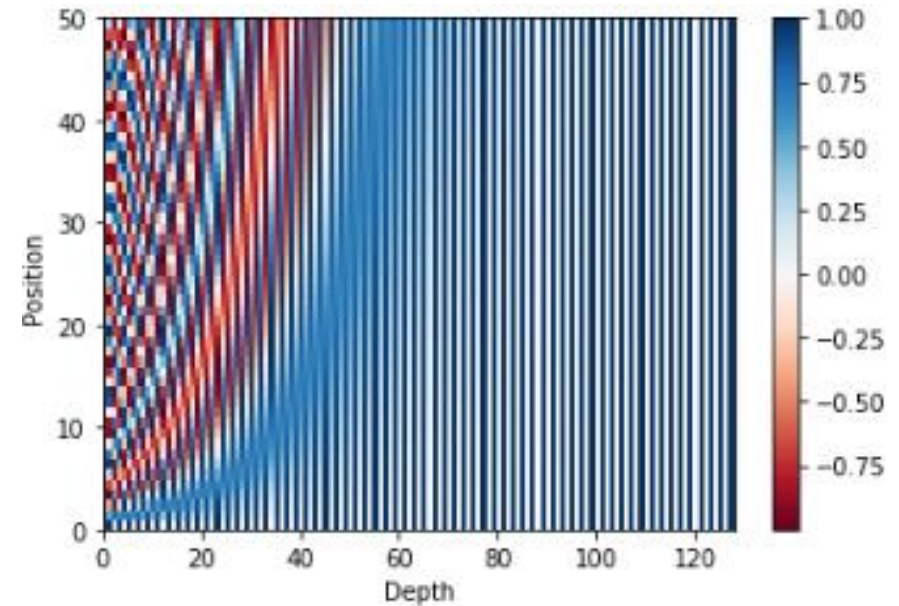
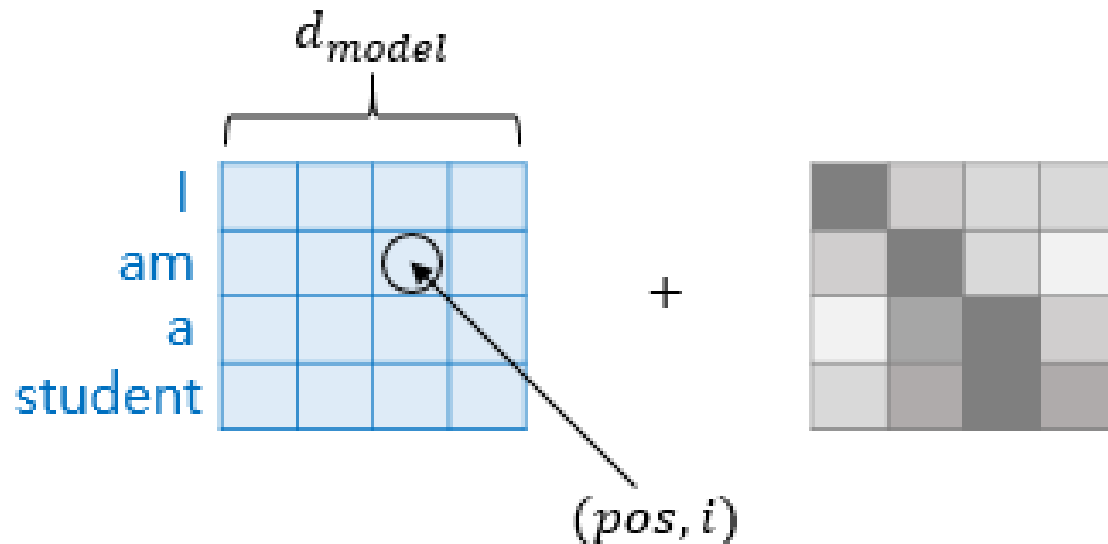
Transformer Implementation

- RNN은 단어의 위치에 따라 순차적으로 입력 받아서 처리
- 트랜스포머는 **포지셔널 인코딩(Positional Encoding)** 사용



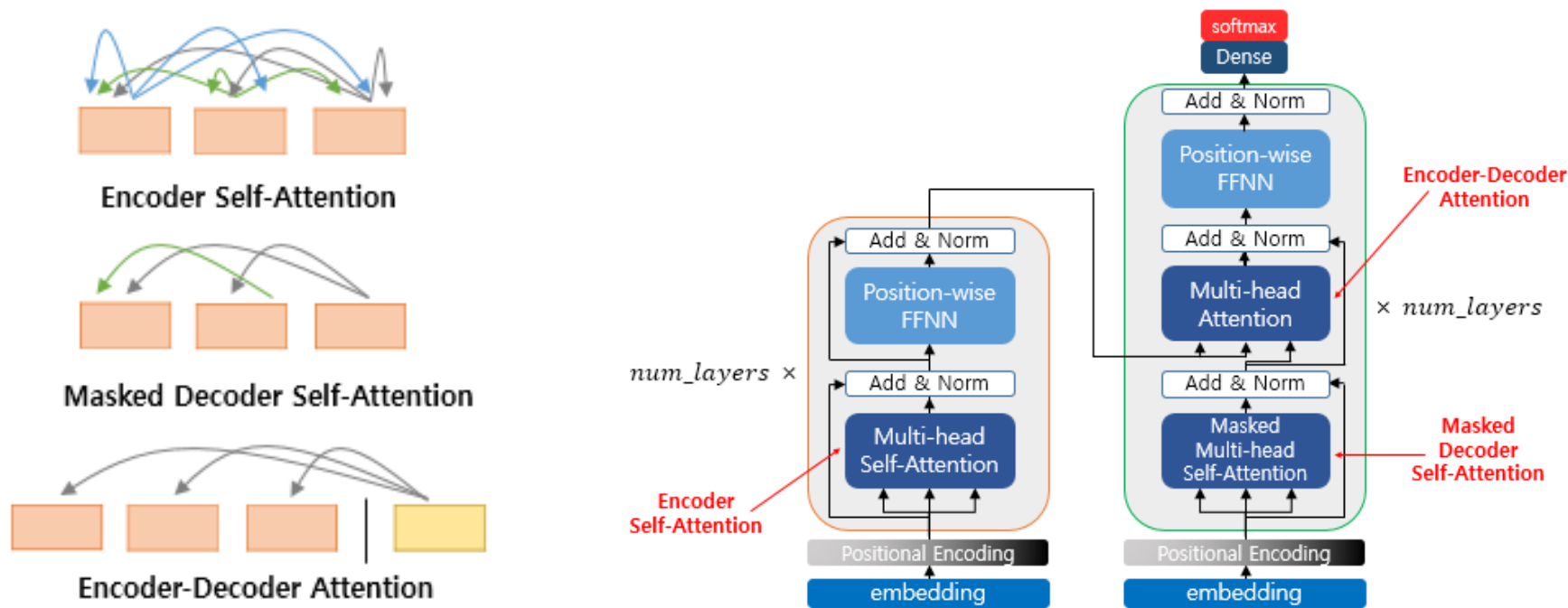
Transformer Implementation

- $PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$
- $PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$
- pos : 입력 문장에서의 임베딩 벡터의 위치 (batch_size, seq_len, d_model)
- i : 임베딩 벡터 내의 차원의 인덱스 (batch_size, seq_len, d_model)



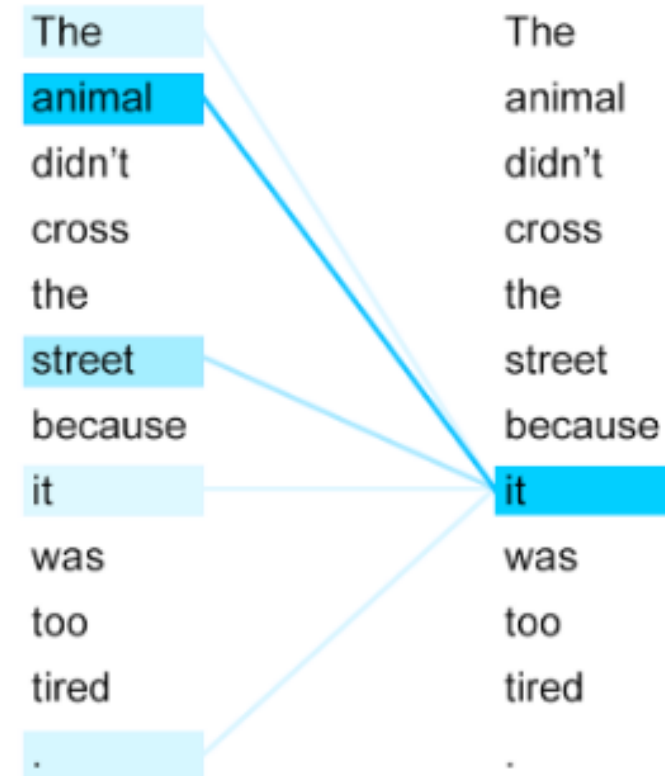
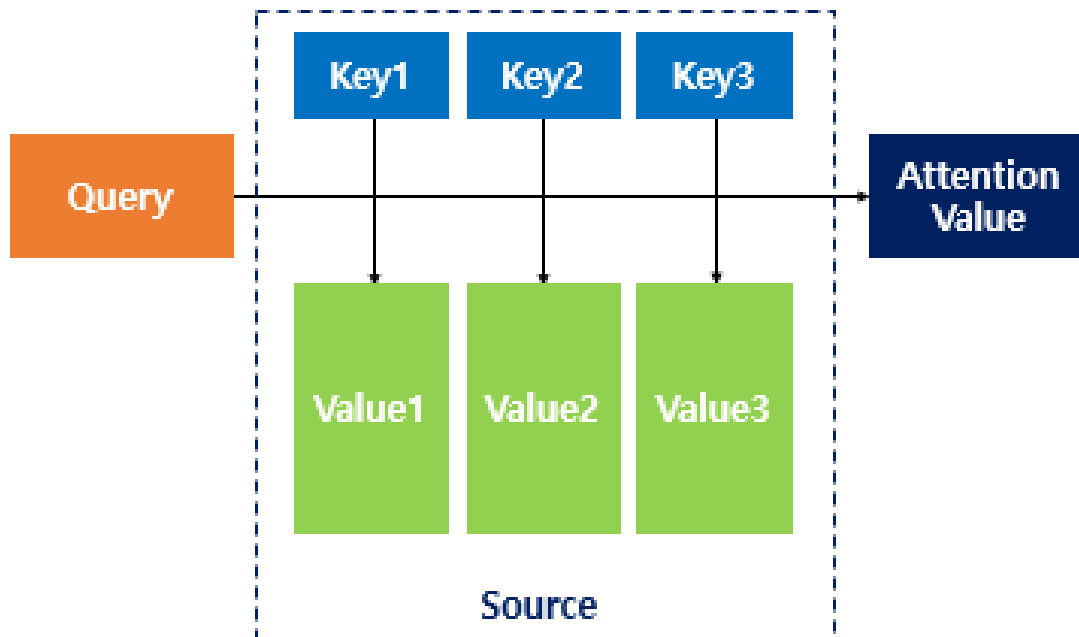
Transformer Implementation

- 세 가지 어텐션 (= 기호는 벡터 값 자체가 같다는 것이 아니라 **벡터의 출처가 같다는 것**을 의미)
 - Encoder Self-Attention: Query = Key = Value = 인코더 벡터
 - Masked Decoder Self-Attention: Query = Key = Value = 디코더 벡터
 - Encoder-Decoder Attention: Query = 디코더 벡터 / Key = Value = 인코더 벡터



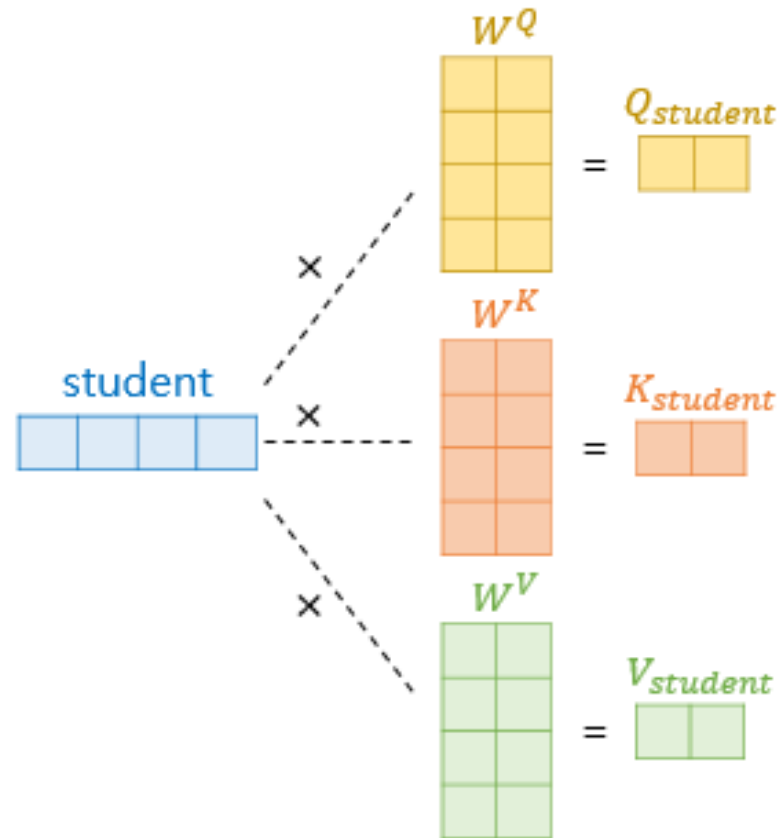
Transformer Implementation

- 주어진 Query에 대해 모든 Key와의 유사도를 구함
- 구한 유사도를 가중치로 각 Key에 해당하는 Value를 가중합



Transformer Implementation

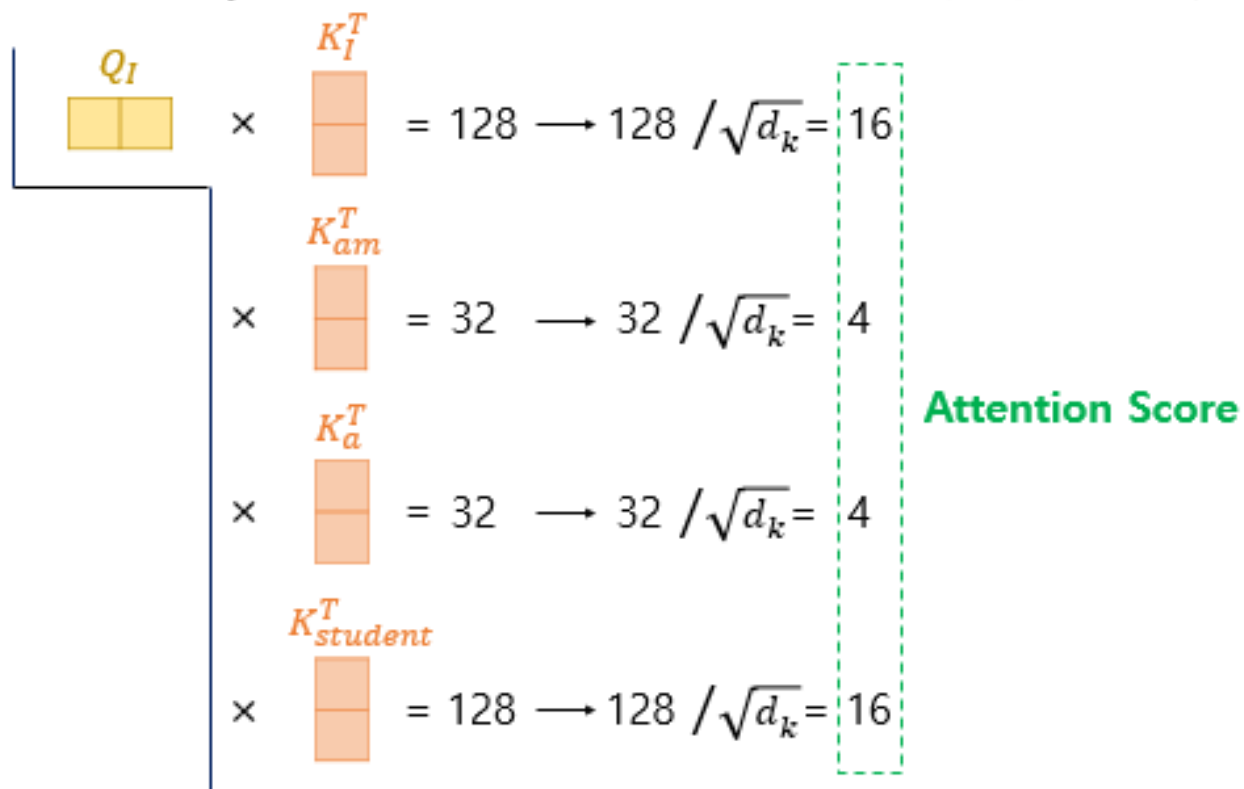
- 각 벡터에 3개의 서로 다른 **가중치 행렬을 곱하여** Q, K, V 벡터를 구함
 - $d_{model} = 512$ 고 $num_heads = 8$ 이라면 $d_k = 64$



Transformer Implementation

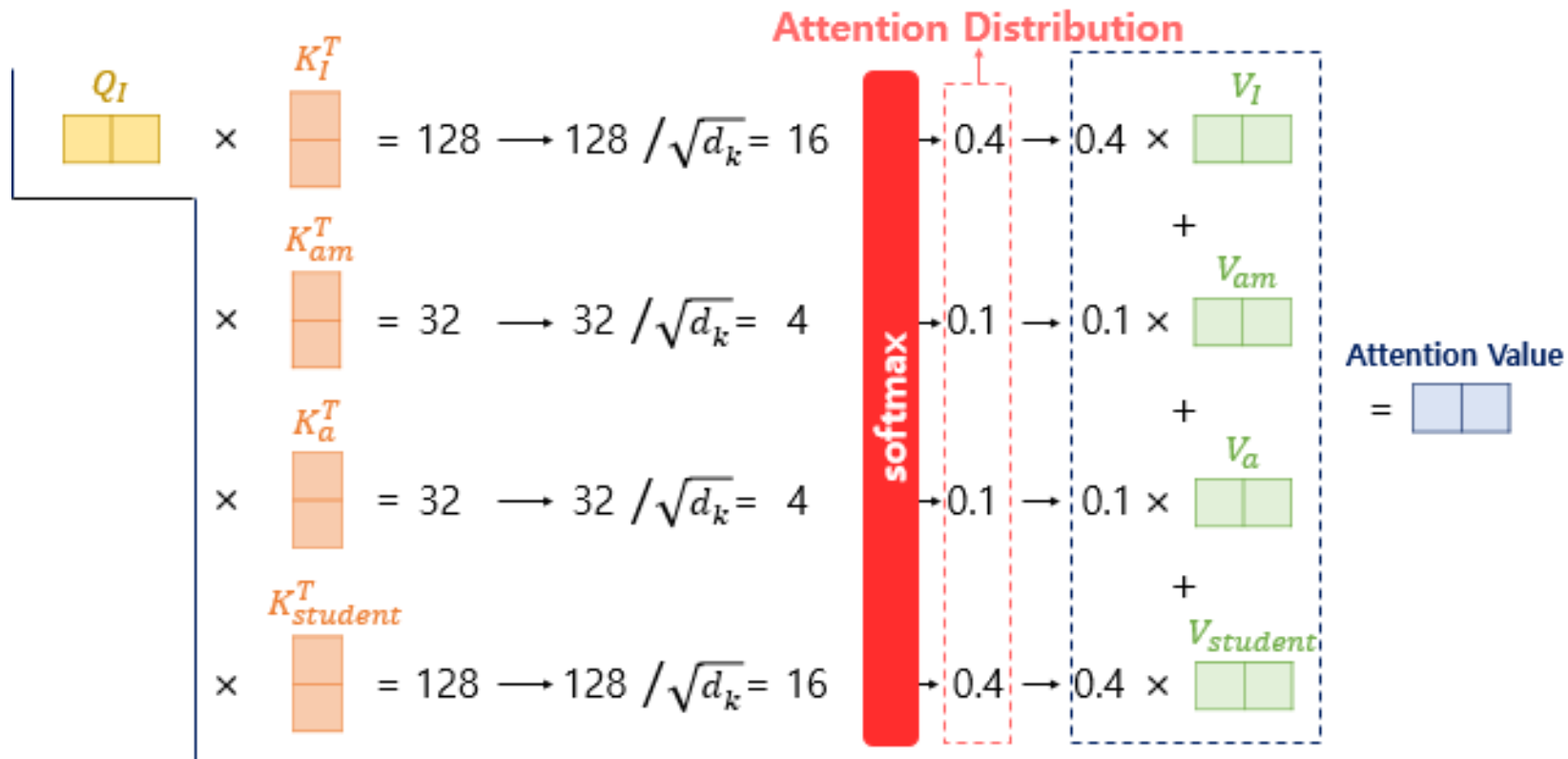
- $score(q, k) = q \cdot k / \sqrt{d_k}$

Scaled dot product Attention : $score\ function(q, k) = q \cdot k / \sqrt{n}$



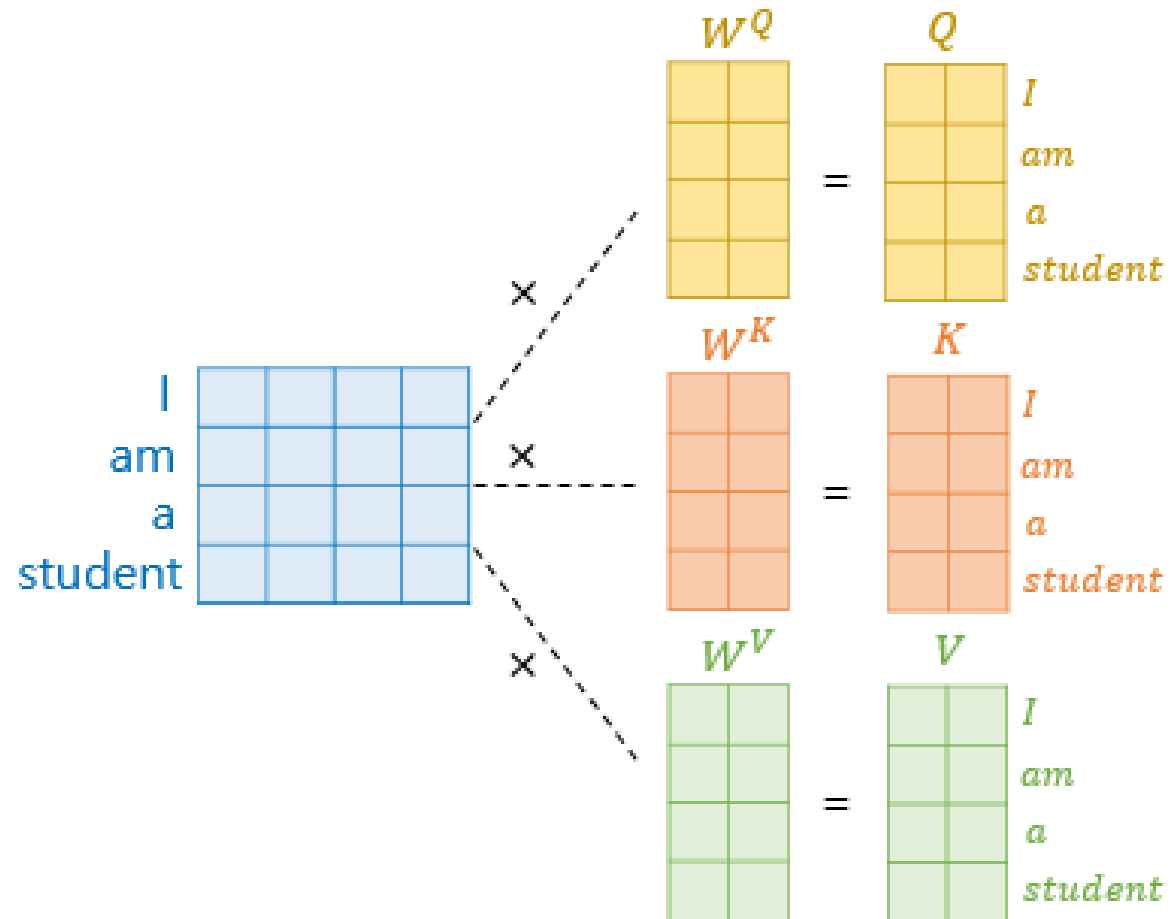
Transformer Implementation

- 어텐션 스코어에 소프트맥스 함수를 적용하여 어텐션 분포(Attention Distribution)를 구함
- 각 Value 벡터를 어텐션 분포만큼 가중합하여 컨텍스트 벡터(Context Vector)를 구함



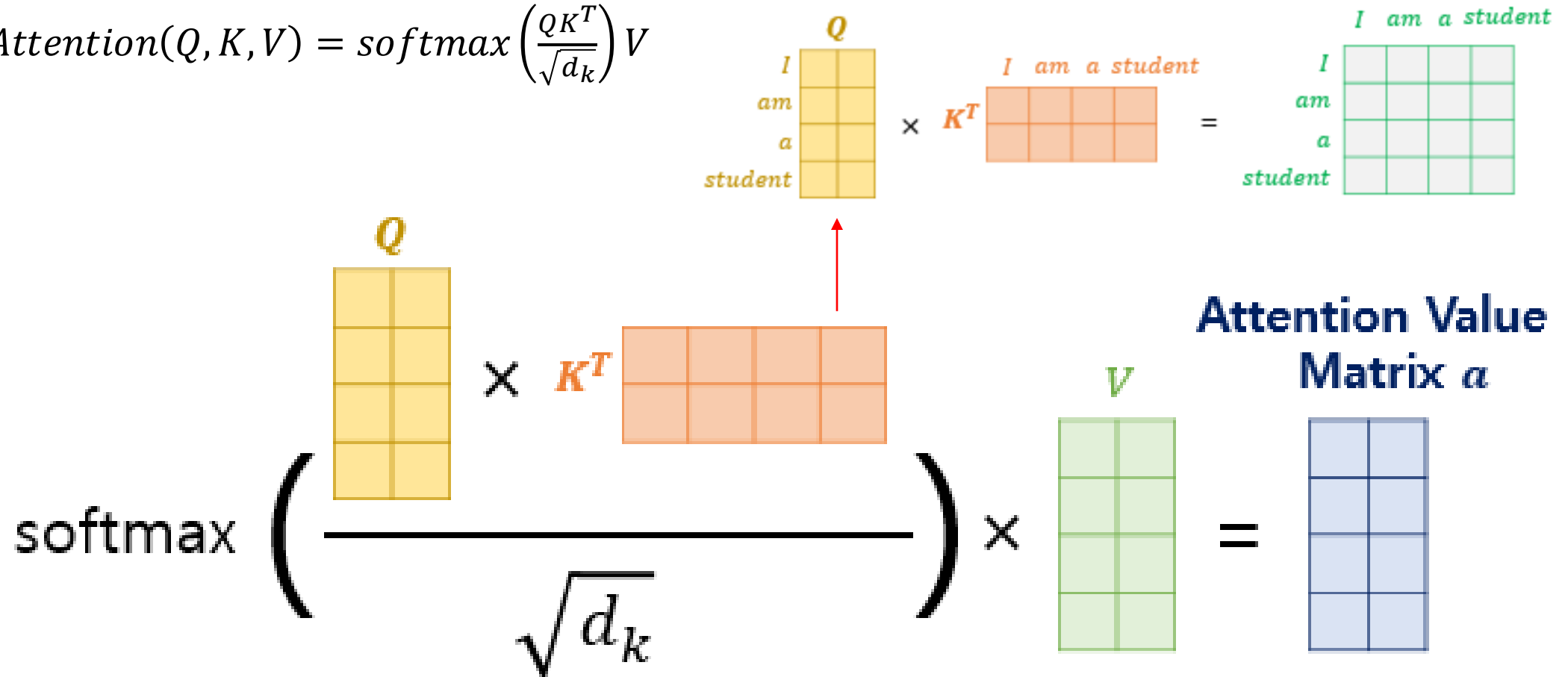
Transformer Implementation

- 각 단어 벡터마다 일일이 계산하는 것이 아니라 문장 행렬에 대한 **행렬 연산으로 일괄 처리**



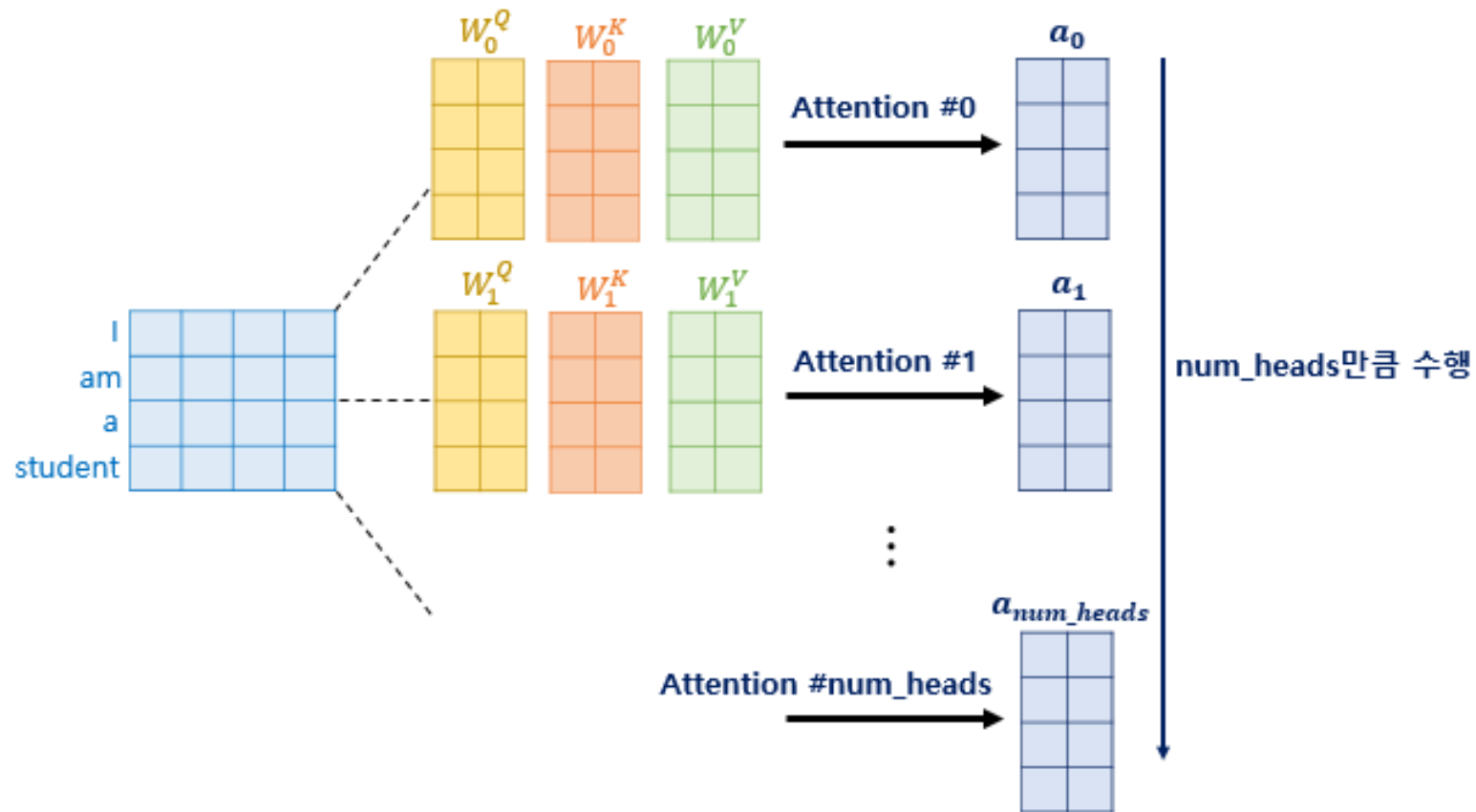
Transformer Implementation

- $Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$



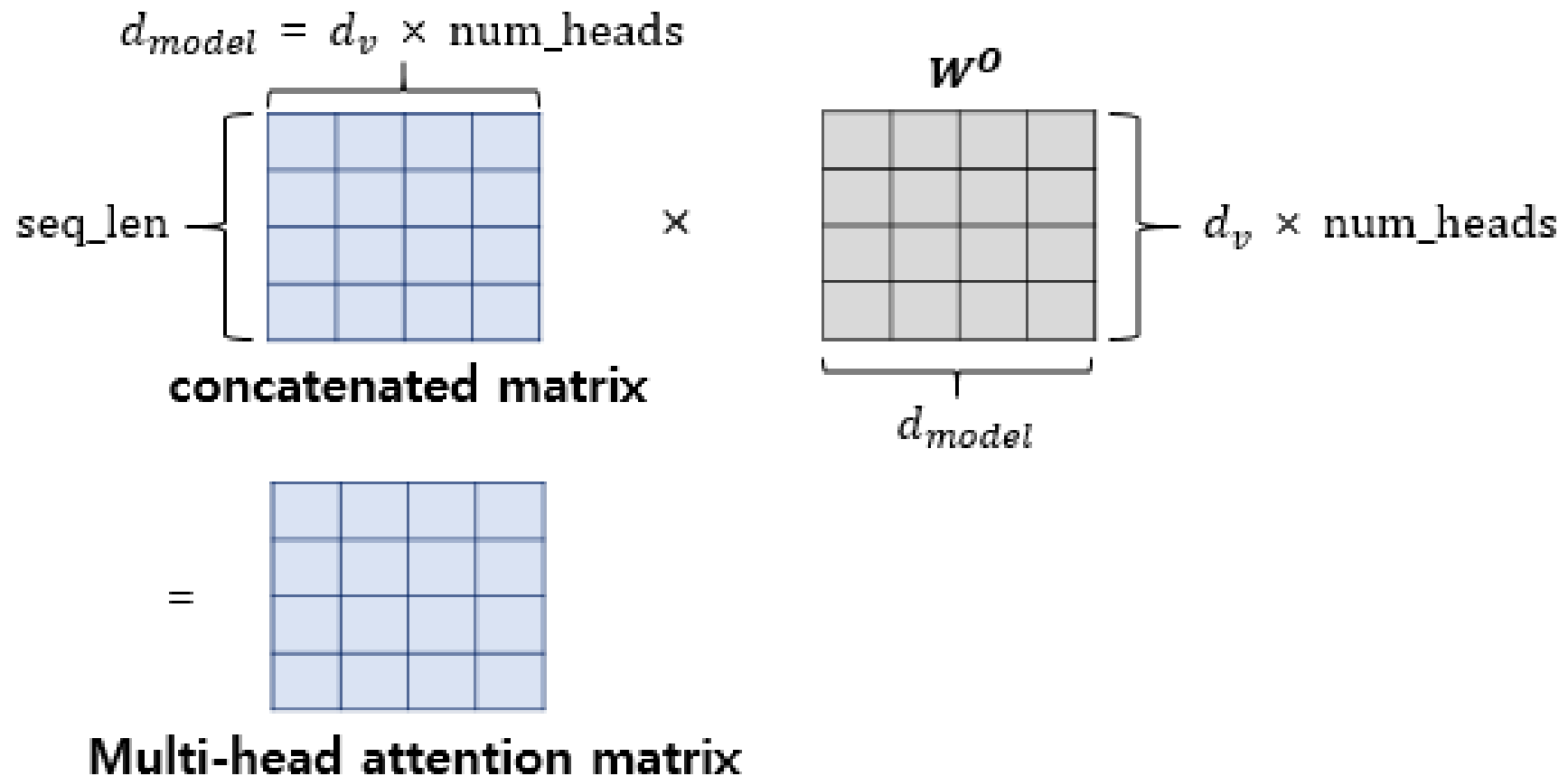
Transformer Implementation

- num_heads 개의 병렬 어텐션
- 어텐션 헤드마다 가중치 행렬 W^Q, W^K, W^V 의 값이 모두 다름



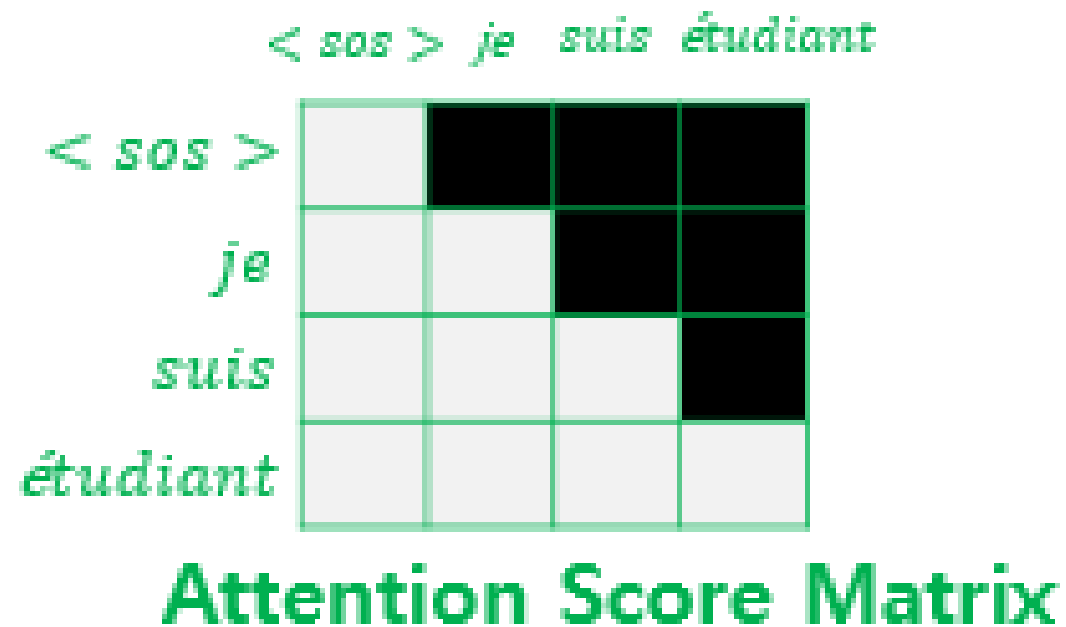
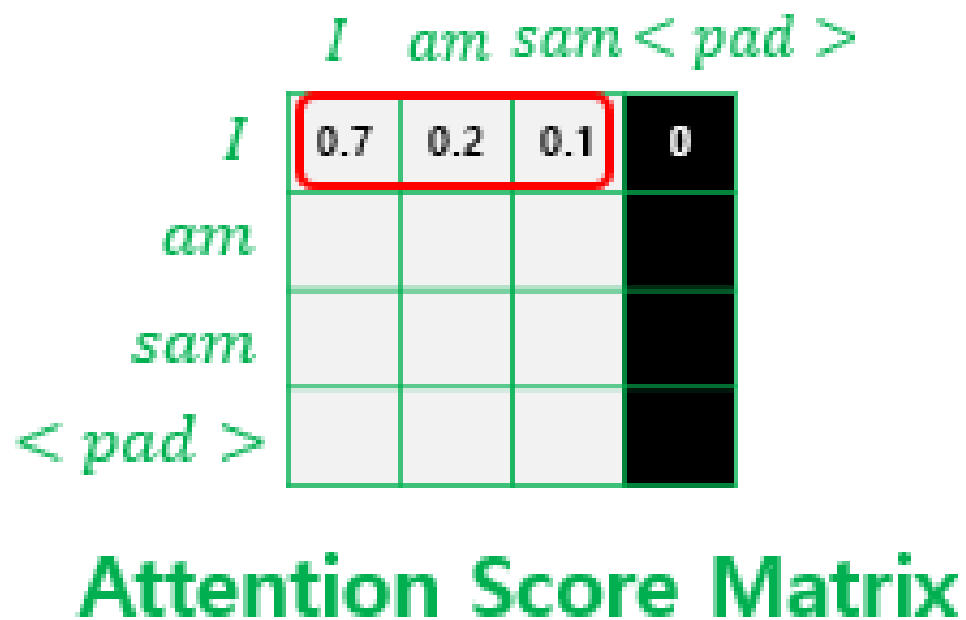
Transformer Implementation

- 어텐션 헤드를 모두 연결(concatenate)하고 가중치 행렬 W^O 를 곱함



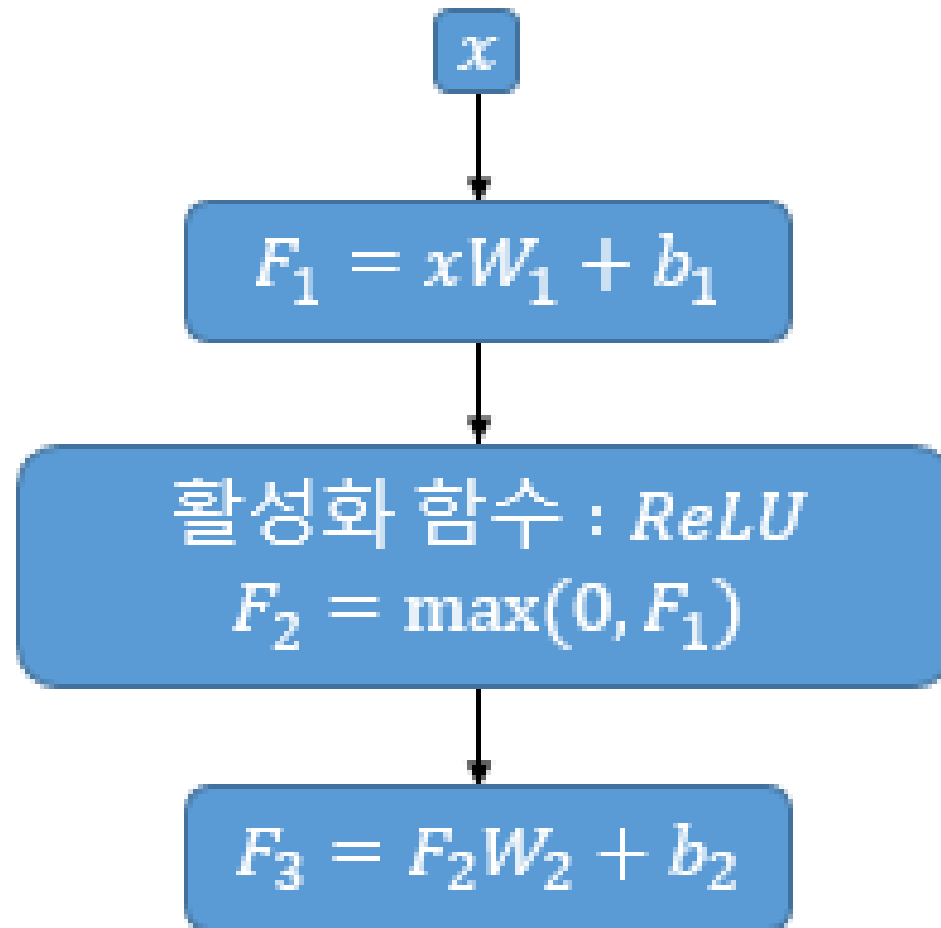
Transformer Implementation

- <PAD> 토큰에 대해서는 유사도를 구하지 않도록 **마스킹 (패딩 마스크)**
- 디코더 셀프 어텐션은 자기 자신보다 **미래**에 있는 단어들을 **마스킹 (룩-어헤드 마스크)**
- 어텐션 스코어 행렬의 마스킹 위치에 **매우 작은 음수값**을 넣어 소프트맥스 후에 0이 되도록 처리



Transformer Implementation

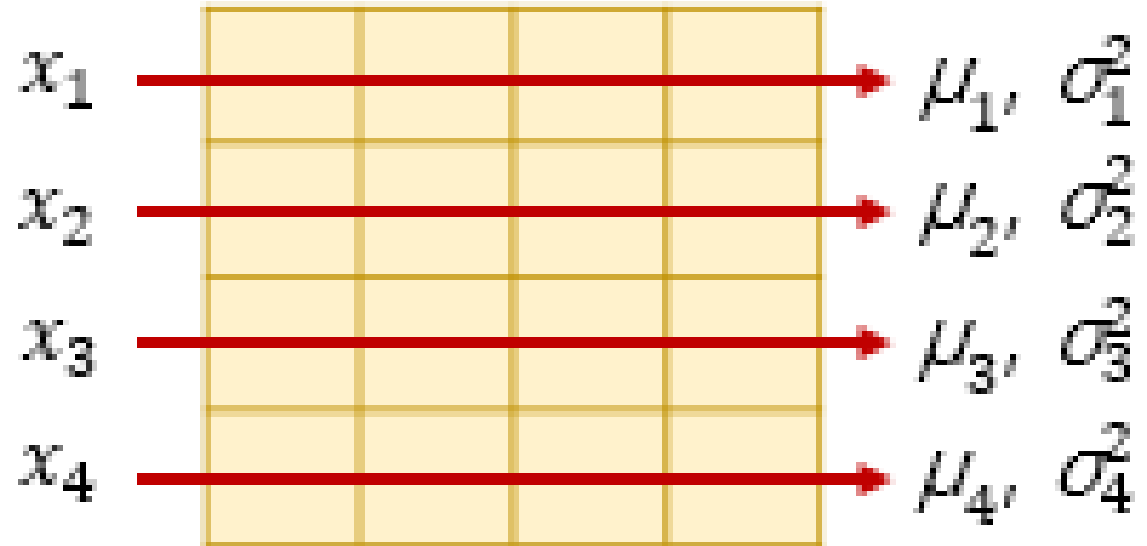
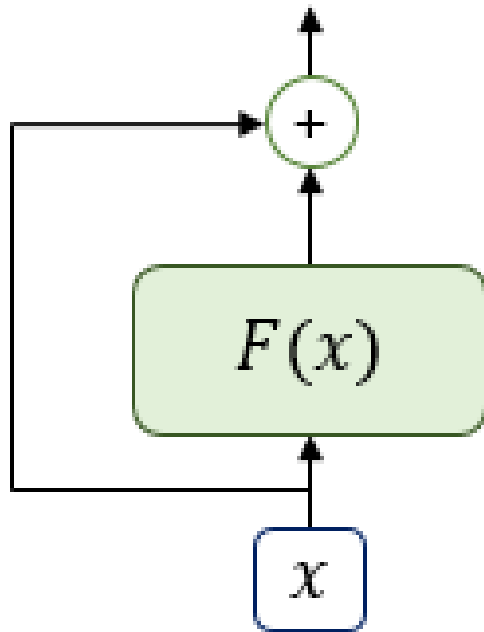
- $FFNN(x) = ReLU(xW_1 + b_1)W_2 + b_2$



Transformer Implementation

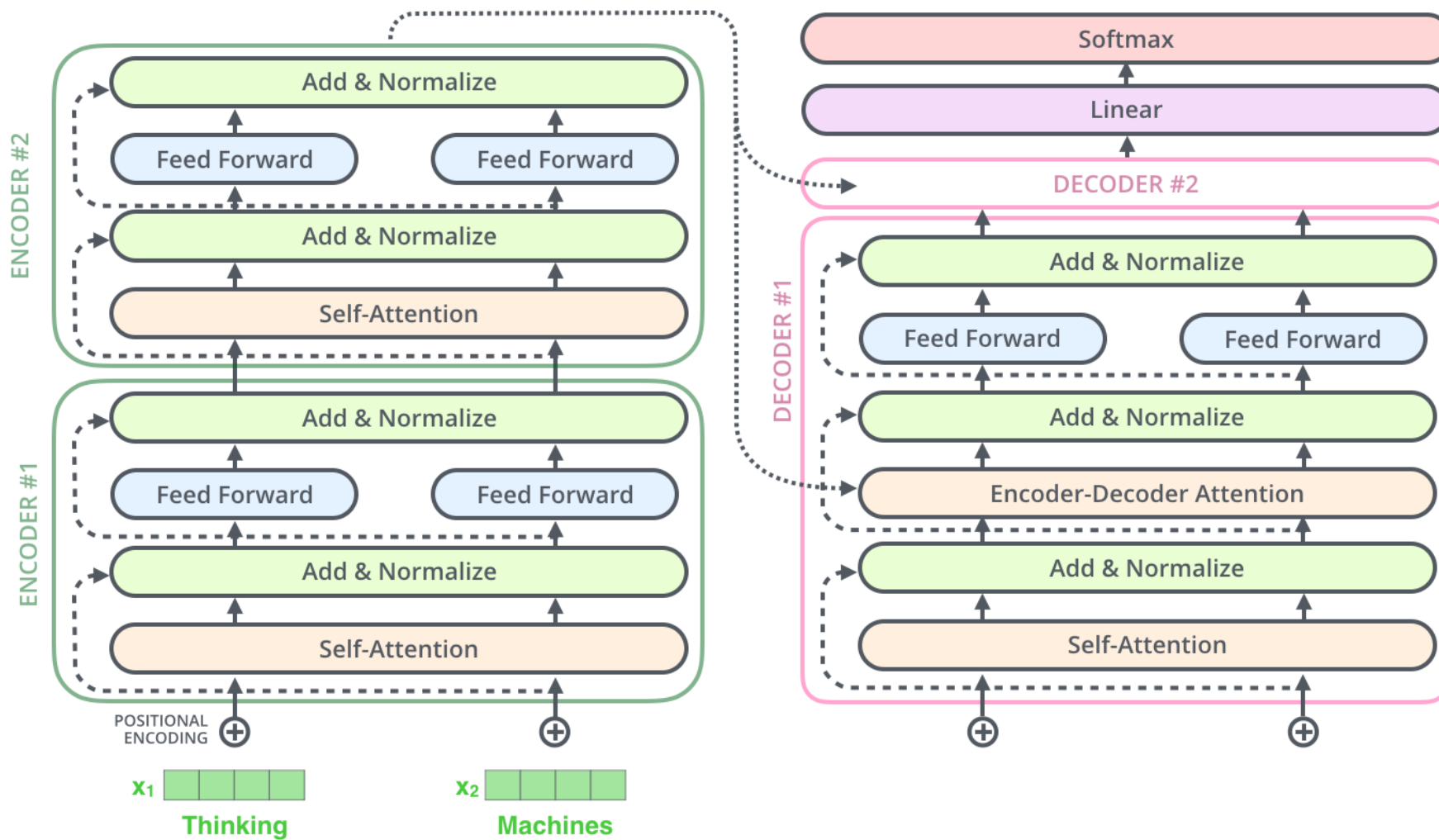
- 잔차 연결(Residual Connection)과 층 정규화(Layer Normalization)

$$H(x) = x + F(x)$$

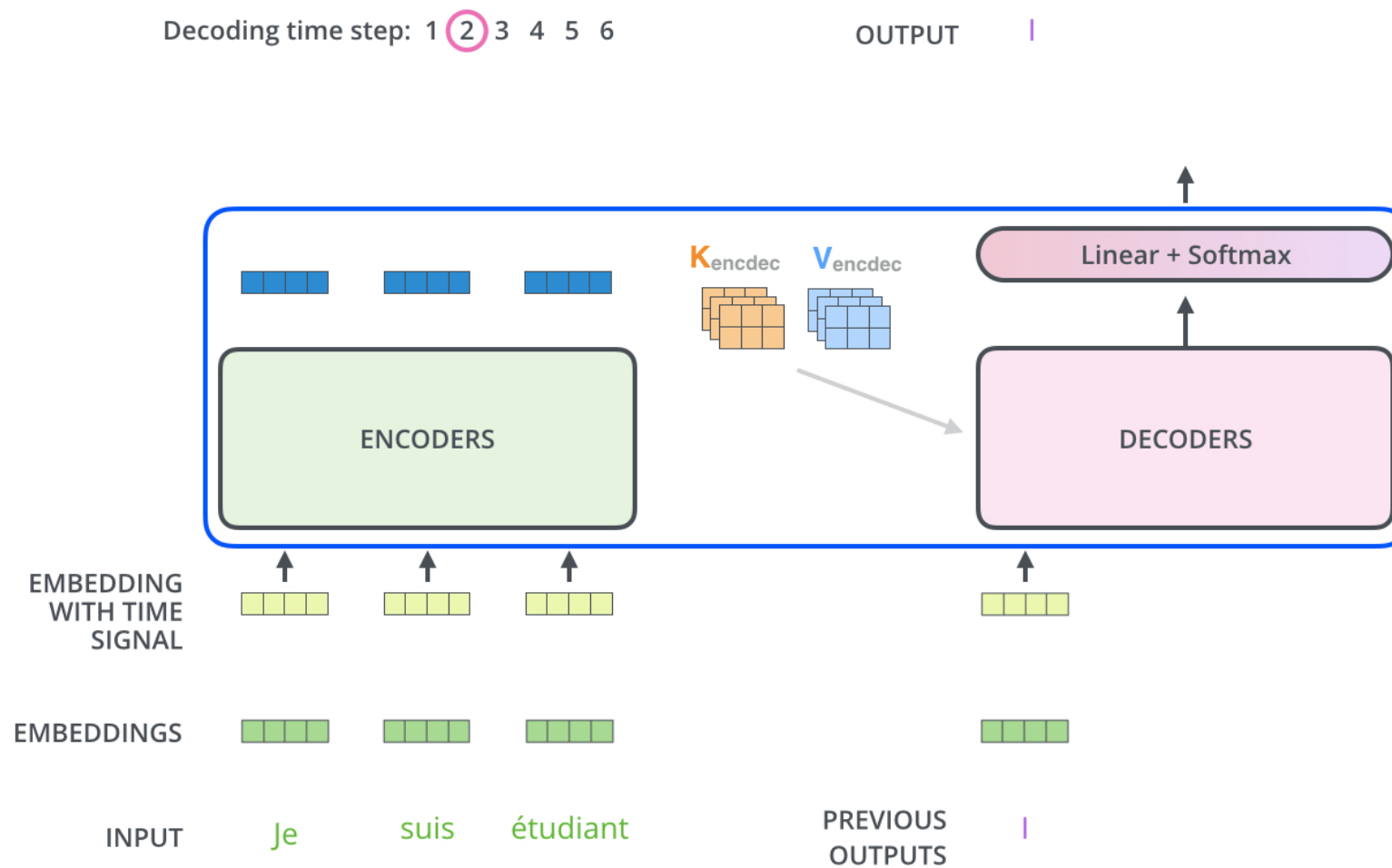


Residual Connection output

(batch_size, seq_len, **d_model**)



Transformer Implementation



Transformer Implementation

Positional Encoding

- [`torch.arange`](#)
- [`Tensor.unsqueeze`](#)
- [`torch.exp`](#)
- [`math.log`](#)

Attention

- [`torch.matmul`](#)
- [`math.sqrt`](#)
- [`F.softmax`](#)

Transformer Implementation

Multi-Headed Attention

- [Tensor.view](#)
- [Tensor.transpose](#)
- [zip](#)
- [Tensor.contiguous](#)

Feed-Forward

- [nn.Linear](#)
- [F.relu](#)

- [The Annotated Transformer](#)
- [The Illustrated Transformer](#)
- [딥 러닝을 이용한 자연어 처리 입문](#)
 - ▶ [16-01 트랜스포머\(Transformer\)](#)

Thank you

Seoul National University

Dong Hyeok Lee

